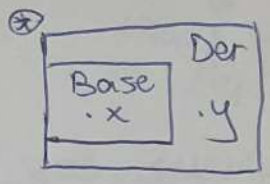
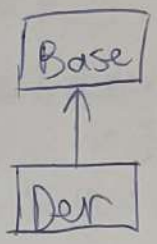
Тема 10

• Der е наследник на Base, ако разширява неговите данни и поведение

class Base { int x; };
class Der : Base { int y; };





Der има член данна от тип Base.

композиция: "has a" relationship Car { Engine }
наследяване: "is a" relationship Car : Vehicle

• видове наследяване

→ public: (най-често)
(Base) (Der)
public → public
protected → protected
private → няма достъп

→ protected:
public → protected
protected → protected
private → няма достъп

→ private:
public → private
protected → private
private → няма достъп

• default наследяване (class Der : Base { };)
→ за struct – public
→ за class – private

• указатели и референции от базовия клас можем да насочваме към обекти на наследника, понеже Base и Der* заемат на едно и също място в паметта и Der притежава всички функционалности на Base (но така basePtr няма да може да достъпи ЧД на Der)

Der d;
Base* bPtr = &d; // V
Base& bRef = d; // V

Base b;
Der* dPtr = &b; // X

f(const Base*) {...}

↓

може да приеме указател към Base или всеки един от наследниците му като аргумент

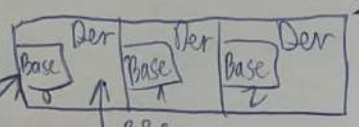
f(const Der*) {...}

↓

не може да приема Base* като аргумент, понеже ф-ята може да има Der-specific behaviour, който не присъства в Base

f(const Base* arr, size) {...}

Der d[3] →



→ компилира се, но е грешно

arr[0]

???

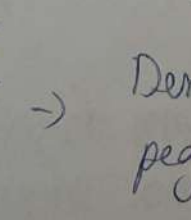
arr[1] → ще се отмести със sizeof(Base), но понеже сме подали Der[], няма да сочи към следващия Base ☹

• конструктори при наследяване

→ Der трябва да каже кой конструктор на Base да се извика; ако не го направи → defaultен; ако няма defaultен → не се компилира

→ Base() се изпълнява преди Der() и конструкторите на ЧД на Der

class Der: Base {
    A a;
    B b;
    C c;
};



→ Der(...): Base(), A(), C()

няма значение

ред на изпълнение:

Base() → A() → B() → C() → Der()

→ аналогично при деструкторите; редът на изпълнение се обръща

~Der() → ~C() → ~B() → ~A() → ~Base()

...ране при наследяване

к.к. и оп= на Der се приемат само за ЧД на Der

```
Der(const Der& other) : Base(other){ copyFrom(other); }
```

логика за копиране на Base — логика за копиране на Der-specific ЧД

```
Der& operator=(const Der& other) {
    if(this != other) {
        Base::operator=(other); //извикваме експлицитно
        free();
        copyFrom(other); // само за Der
    }
    return *this;
}
```

- аналогично за тази също викаме т.с. и т.оп= на Base експлицитно

→ Всеки клас се грижи само за неговата динамична памет

```
class Base {
    char* str;
    //б-ка
};
```

```
class Der:public Base {
    int num;
    // НЕ разтриваме б-ката тук
};
```